NyayaTrace — treatment-aware precedent search for Indian case law

CSD358 Information Retrieval · Mid-term Hackathon · Track T6 — vertical search for law

Rakshit · Suvrat · Sumedha

Every number in this report is measured, and there are no placeholders left. The sources are `runs/trackA.json`, `runs/citation_recovery.json`, `runs/efficiency.json`, `runs/operators.json`, `runs/ablations.json`, `runs/tuning.json` and `runs/corpus_stats.json`, each produced by the command named beside it in Appendix A. Nothing here is estimated, rounded in our favour, or carried over from a pilot run. Where a measurement moves between runs we give a range rather than a figure, and where a number came from a harness arm rather than the shipped code path we say which.

1 Problem and track relevance

A lawyer looking for precedent is not asking "which document is about privacy". They are asking "which binding judgment, still good law, decided the point I am arguing, and how have later benches treated it". A general-purpose retrieval system answers the first question and is silent on the second, and the second is the one that decides a case.

Three properties of Indian Supreme Court judgments make this a *vertical* problem rather than a corpus to point an off-the-shelf index at.

The vocabulary is a citation system, not prose. The same provision appears as *Section 302 IPC*, *S. 302 of the Indian Penal Code, 1860*, and *u/s 302*. To a standard tokenizer those three strings share almost no terms, so one provision is scattered across three postings lists and a query for any one of them misses the other two. The problem is live rather than historical: the Penal Code was replaced by the Bharatiya Nyaya Sanhita on 1 July 2024 and the section numbers moved, so a corpus spanning 1950–2025 contains both numbering systems for the same offence.

A judgment is a structured document. It has a cause title, parties, a bench, the statutes it turns on, an operative order, and a very long body of facts, submissions and reasoning. **84.8%** of the 83.7 million tokens we index sit in the body zone, and the statutes zone holds 0.1% of them. A flat bag-of-words model therefore spends almost all of its evidence on recital, and the holding — the part a lawyer actually wants — is drowned by it. The bench, meanwhile, is not evidence of relevance at all: it is a *filter*, because a five-judge bench binds a three-judge bench.

Citation count and reliability come apart. A judgment can be cited hundreds of times and still be the one case you must not cite, because a later and larger bench overruled it. Every earlier citation still points at it, and no link-popularity measure ever retracts a vote. Text relevance cannot detect this, and neither can plain PageRank; both will rank a dead precedent at the top.

NyayaTrace is built around these three. It is squarely Track T6 because every component exploits domain structure a general web engine has no access to: zones derived from the shape of a judgment, parametric metadata (year, bench size, judges, acts, case type) that supports real filters, a citation graph whose edges carry a *legal* relation rather than a hyperlink, and an authority score that is a statement about precedential standing rather than about popularity.

Corpus. We use the AWS Open Data release of Indian Supreme Court judgments (CC-BY-4.0): **42,371** English judgment rows spanning **1950–2025**. De-duplicating on the bucket path removes 5,161 of them — 12.2% of the release is the same judgment scraped once per language — leaving **37,210** distinct judgments, which are the nodes of the citation graph and the source of every parametric fact.

We index the *text* of a **stratified sample**: at most **200 judgments per year** (seed 358, so the draw is reproducible), taking years with fewer judgments whole. The manifest holds 13,942 documents and **13,878 were extracted successfully (99.54%)**; the 64 drops are 53 scans, 7 documents under the word floor, 2 not in English and 2 missing from the bucket, each with its reason in `extract_log.csv`.

The indexed subset averages **6,030 tokens** (median 4,161) and **35.3 paragraphs** per judgment, for **489,627 paragraphs** in all. Equal temporal coverage means the whole history of the Court is searchable rather than only the last decade. The sampling rate is a known limitation and we treat it as one — see §6.

2 How we used IR

The system is one code path with switchable layers. Figure 1 shows the three horizontal bands: what runs at build time, what sits on disk, and what happens when a query arrives.

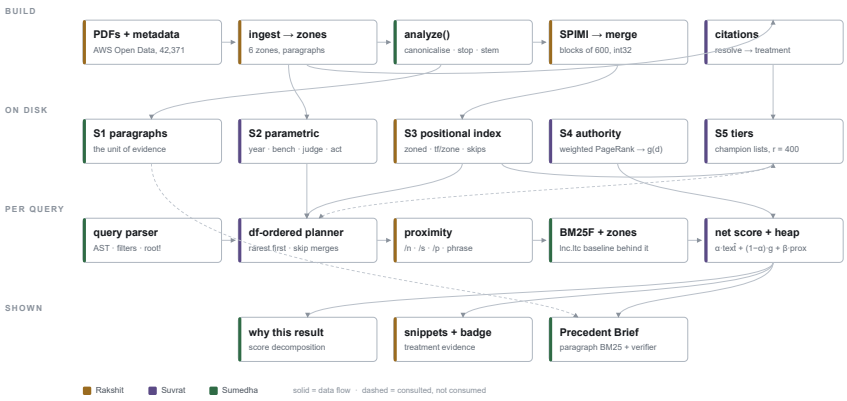


Figure 1 — the NyayaTrace pipeline. The build row runs once; the stores row is what exists on disk (S1–S5 of the team contract); the query row is what executes per search; the bottom row is what the user is shown. Colour is ownership, which is also how the work was divided (§7). Note that `analyze()` is the only path into the index *and* the only path into a query, which is the invariant the whole system is built on.

2.1 Ingest and zoning

PDFs and the metadata parquet files are pulled from the bucket, text is extracted page by page, and a page yielding fewer than 200 words is treated as a scanned image, with the document logged as a drop rather than silently kept. Every retained judgment is cut into the six zones of `config.ZONES` — `title`, `parties`, `bench`, `statutes`, `order`, `body` — and into numbered paragraphs, which are the unit of evidence for snippets, proximity and the Precedent Brief. Each attempted document gets a row in `extract_log.csv` with its drop reason, so the corpus is auditable rather than merely large.

2.2 The analyzer

`analyze(text, zone, para_id)` is the single entry point for documents **and** queries. That is the one invariant we froze before writing anything else: if the two ever diverge, matching breaks silently and no test catches it. It case-folds, tokenises, canonicalises statutes (§3.1), drops a legal stop list — words like *appellant*, *hon'ble*, *judgment* carry no discriminative power when every document contains them — and applies a Porter stem with statute tokens exempt, because `ipc§302` is an identifier and stemming an identifier is a bug. Each emitted token carries its surface form, its running offset within the zone, its sentence id and its paragraph id.

2.3 Zoned positional inverted index

Built with SPIMI: 600 documents are inverted in memory, flushed to a block on disk, and the blocks are merged once into a single memory-mapped `int32` array. Nothing holds the whole index in RAM and nothing is a Python dictionary at query time. A posting is `(doc_id, {zone → tf}, positions)` — which document, how often in each zone, and exactly where. Every ⌈√df⌉ entries the postings list carries a skip pointer, so a merge between a short list and a long one does not read the long one end to end. The index also stores `doc_len`, `zone_len` per document and the `lnc` vector norms the cosine baseline needs.

Sizes: **544,787** distinct terms (68.6% appearing in exactly one document), **83,685,567** token positions and **11,943,968** (term, document) postings in **687.3 MB** of `postings.bin`, built by SPIMI in **604 s**. The S3 store is 717.9 MB, every store together 1.9 GB, an index-to-text ratio of 1.29. Cold, the stores take 1.2–1.5 s to memory-map and the first query about 1 s; warm queries are 4–7 ms (§5.4).

The collection obeys the usual laws, which checks the tokenizer rather than proving anything: Heaps `|V| = 3.33 · T^0.653` ($R^2 = 0.9957$), Zipf `s = 0.9825` ($R^2 = 0.9930$) over ranks 10–1,000. Document frequency spans four orders of magnitude — `surveil` 58, `privaci` 126, `const§21` 623, `ipc§302` 1,081, `court` 13,877 of 13,878 (idf 0.000, so a stop word).

2.4 Parametric index

A separate store over year, bench size, judge names, acts, case type and disposal, exposing `select(field, op, value) → sorted doc ids`. This is what makes `bench:>=5` and `year:>=2000` genuine constraints rather than keyword matches against the bench zone, and it is why the bench zone has weight 0 in ranking: scoring it as well would be double counting.

2.5 Query language and parser

A Westlaw-flavoured syntax the profession already knows: `AND OR NOT`, quoted phrases, `/n` (within *n* words), `/s` (same sentence), `/p` (same paragraph), root expansion `bail!` by dictionary prefix scan, zone restriction `title:(privacy)`, and filters `year:>=2000 bench:>=5 judge:chandrachud act:IPC`. The parser returns an AST of `TermNode / PhraseNode / ProxNode / AndNode / OrNode / NotNode / FilterNode` plus, for a query with no operators, a plain bag of terms that takes the free-text fast path.

2.6 Boolean planning and merges

The planner never executes a conjunction in the order the user typed it. It orders the operands by document frequency and processes the rarest first, because an intersection can only shrink: starting small keeps every later merge small. Selective parametric filters are folded in first for the same reason. All intersections use the skip pointers. Index elimination drops query terms below an idf of 0.35, and for a free-text query of four or more terms only documents matching at least half of them are scored at all.

On the worked example the planner is handed `privacy /p "fundamental right" AND bench:>=5 year:>=2015` and chooses `PROX_P[126] → FILTER(bench≥5)[1154] → FILTER(year≥2015)[2139]`, walking the candidate set down **126 → 66 → 16 → 9**. The planner's own A/B prints the cost: **3,375** postings entries compared in that order against **3,449** as written, for an identical answer. The margin is small on this query because the proximity node is already the rarest operand; the ordering exists for the queries where it is not.

So we measured those queries too, and the whole experiment is one command: `python -m nyaya.query.planner --compare-suite` replays eight AND chains under three operand orders — df-ordered (shipped), filters-first, and exactly as the user typed it — and counts the postings entries each one compares. Over the eight: **145,921 → 69,527 entries compared, a 2.10× reduction**, and **identical answers under all three policies on all eight queries** — this is a lossless reordering, not an approximation. Best case **327×**: `tax`

`AND sale AND interstate AND trade` compares **29** entries df-ordered against **9,490** as written, because `interstate` is rare enough to collapse the chain on the first merge. Worst case **1.00×**, on `bail AND anticipatory AND discretion`, which happens to be written cheapest-first already. The 2.10× is computed on the *totals*, not as a mean of per-query ratios, so the 327× outlier cannot carry it. That is the lectures' "process operands in order of increasing document frequency" measured on the real index.

2.7 Proximity

Because every token carries its offset, sentence id and paragraph id, `/n`, `/s` and `/p` are one two-pointer merge over two position lists with three different comparisons: `|pos_a − pos_b| ≤ k`, `sent_a == sent_b`, `para_a == para_b`. A quoted phrase is the ordered, tightest case of the same walk. The merge is linear in the two lists and never materialises a cross-product.

2.8 Ranking, as switchable layers

1. **lnc.ltc cosine** — the textbook tf-idf baseline: flat document, no zones.
2. **Weighted zone scoring** — a per-zone score combined with `statutes 2.0, order 1.5, body 1.0, title 0.5, parties 0.5, bench 0`. The `title` weight is 0.5 rather than the textbook 3.0 on measured evidence: a Supreme Court cause title is a *party name*, not a subject line, so a query term almost never appears in one — 1 of the 126 candidates for our worked query had a title hit — and a per-zone cosine over a nine-token title amplifies that single hit enormously. S2's nDCG@10 falls monotonically as the title weight rises, on both splits; BM25F is insensitive to it, so the change fixes S2 and costs the shipped S4 nothing — see §5.6 for the evidence and §2.11 for what the tuner made of it.
3. **BM25F** — the per-zone term frequencies are combined *before* saturation, each with its own length normaliser `b_z`, and the sum is then saturated with `k₁ = 1.2`. Saturating per zone and adding afterwards would let a long body out-shout a title, which is the whole reason BM25F exists rather than BM25 over a concatenated document.
4. **Proximity contribution** — the tightest window the operators found, inverted.
5. **Net score** — `net = α·text̃ + (1−α)·g(d) + β·prox` with `α = 0.80`, `β = 0.05`, where `text̃` is min–max normalised within the candidate set and `g(d)` is the static authority of §3.4.

Setting `α = 1` removes authority; swapping the scorer returns the cosine baseline. Each of the six evaluated systems is one flag away from its neighbour.

2.9 Efficiency structures

Champion lists hold the top 400 documents per term by a static quality measure, and a two-tier index serves the free-text fast path from tier 1, falling back to the full postings only when tier 1 cannot fill *k*. Results are collected through a heap rather than a full sort. Measured effect in §5.4.

2.10 Tuning — and why we did not ship the tuned values

Coordinate ascent over the zone weights, α and β: two passes, **57 configurations**, fitted on the **five held-out dev queries only** (the tuner's CLI refuses `--split test`, so the test set cannot be fitted on by accident). Range: title [0 … 3], parties [0 … 1], statutes [0 … 4], order [0.5 … 3], α [0.60 … 1.00], β [0.00 … 0.20].

It moved neither α nor β — the shipped 0.80 and 0.05 are the grid optimum. Its one proposal, `statutes 2.0 → 0.0` with `order 1.5 → 1.0`, gained **+0.0045 dev nDCG@10** and cost **−0.0200 on the 20 test queries**, so **we did not ship it** and `config.py` records why. A tuner that improves its own fitting set while losing on held-out data is doing the one thing a tuner must not do; five queries cannot support a weight change. A rejected ablation is as much a result as an adopted one, and it would have been easy, and dishonest, to ship the tuned weights and quote the dev gain.

2.11 Explanation

Every result carries a `ScoreExplanation` dataclass: raw and normalised text score, per-zone contributions, top terms by idf·tf, proximity window, `g(d)` and its `pagerank` / `bench` parts, the status penalty, α, β and the net. Because it is a dataclass produced by the scorer rather than a string assembled for display, it cannot drift away from what was actually computed.

---

3 Beyond IR

Five components are not retrieval techniques; they are domain engineering that retrieval then exploits.

3.1 Statute canonicalisation and the IPC↔BNS crosswalk

A grammar over Indian statutory citation forms — not a phrase list — recognises `Section 302 IPC`, `S. 302 of the Indian Penal Code, 1860`, `u/s 302`, `sec. 302 I.P.C.` and their punctuation variants and rewrites all of them to the single token `ipc§302`. It covers the Constitution, IPC, CrPC, CPC, the Evidence Act and the three 2023 replacement codes, handles sub-sections and clauses, and runs once at build time so it costs nothing per query. It is also the single most valuable component we built (§5.6). On the query side a crosswalk expands `ipc§302` to fire `bns§103` as well, because the corpus straddles the 2024 recodification; 120 crosswalk rows are verified against the official Government of India correspondence tables, and the expansion is a flag so S1 can measure canonicalisation alone.

The canonicaliser scores **92.5% precision and 95.9% recall** on named known acts over 151 real judgments, and statute tokens reach **10,924 of 13,878 documents (78.7%)** — 61,314 tokens across 110 distinct acts.

3.2 Citation extraction and resolution

Judgments cite each other in prose. We extract candidate citations and resolve them to node ids in three passes: neutral citation (`2017 INSC 1136`), SCR citation (`[2017] 11 S.C.R. 1029`), and case-name matching requiring a Jaccard overlap of at least 0.6 over normalised party-name tokens. Each edge records how it was resolved, so an unresolved or name-matched edge can be excluded from an ablation. Result: of **76,958** citation occurrences found in prose across 7,803 judgments, **66.0%** resolved to a node once 840 self-citations are dropped, giving **76,118 edges** onto 11,317 distinct targets. Resolution quality is not uniform and we report it by rung: exact SCR keys carry 46.5% of all occurrences and resolve at 90.8%, while SCC and AIR citations — which give only a name and a year — resolve at 51.0% and 40.6%. The name-matching rung was hand-audited at **91.3% precision**.

3.3 Treatment classification

The novel part. For each citation we take the sentence and the surrounding paragraph and classify the *treatment* the citing court applied, using a cue-phrase classifier over four classes: **followed** ("we respectfully follow", "we are in agreement with"), **distinguished** ("is distinguishable on facts", "turned on its own facts"), **overruled** ("stands overruled", "no longer good law", "is hereby overruled") and **mentioned** (everything else). This is a legal-reasoning label, not a link type, and it is what makes the authority model possible. On 40 hand-labelled edges the classifier is right **92.5%** of the time; §6 is about the three it gets wrong.

3.4 Treatment-weighted authority

The citation graph is weighted by treatment — `followed 1.0, mentioned 0.6, distinguished 0.3, overruled 0.0` — row-normalised, and we run our own power iteration (`λ = 0.85`, 60 iterations, tolerance 1e-10; no graph library). An overruling edge therefore transfers no authority at all. The PageRank is then combined with bench size, which is the Court's own hierarchy of precedential weight, and multiplied by a status penalty:

```
g(d)    = (0.75 · pagerank_norm + 0.25 · bench_norm) × penalty
penalty = 0.25  if overruled
          0.70  if frequently distinguished
          1.00  otherwise
```

A case counts as *frequently distinguished* when at least 34% of its in-edges are distinguishing and it has at least three such edges. **91** judgments in our corpus carry an overruled status and 86 are frequently distinguished; each is flagged in the interface with the citing judgment and the paragraph that did it. An overruled judgment loses about three quarters of its `g(d)`: *Pune Municipal Corporation* (2014), cited 73 times with 17 of those citations saying overruled and itself overruled by *Indore Development Authority* (2020), falls from 0.377 to 0.087. *T. Devadasan* (1964), 3 overruling citations of 61, falls from 0.460 to 0.115.

One caveat is printed beside that list rather than suppressed. **67 of the 91** rest on a *single* overruling citation, and one edge against a hundred that say "followed" is more likely a classifier error than overruled law. Those rows are marked `(!)` rather than filtered, because a share threshold would also discard the genuine single-overruling cases — *Sankari Prasad* is 1 of 73.

3.5 Precedent Brief — retrieval-augmented generation without vectors

An optional feature that drafts a short brief. Paragraph-level BM25 over the same positional index selects at most 8 paragraphs from the top 5 documents, never more than 2 from one judgment. A language model is then required to attach a paragraph id to every sentence it writes. A tf-idf cosine verifier re-tokenises each generated sentence through `analyze()` and scores it against the paragraph it claims; below a cosine of 0.20 the sentence is flagged **unsupported** in the output rather than deleted, so the reader sees the disagreement. There is no embedding model and no vector store anywhere in the system: the retrieval is the system we already measured.

---

4 Novelty and creativity

Treatment-aware authority is the main claim. Link analysis over a citation graph is standard; treating the *label on the edge* as the weight, and separating "how much authority flows" from "is this still good law", is not. They are deliberately different mechanisms: an overruling edge carries weight 0 in the PageRank (it transfers nothing) *and* triggers a multiplicative status penalty on its target (it demotes). A purely spectral method cannot express the second, because a node with no outgoing authority is not the same thing as a node that must be pushed down. The visible consequence is the one a practitioner cares about: the most-cited judgment on a point can be demoted and badged, with the sentence that overruled it shown as evidence.

Statute tokens as first-class index terms. Collapsing three surface forms to `ipc§302` turns a vocabulary-mismatch problem into an exact-match one and makes `act:IPC` meaningful; the IPC↔BNS crosswalk makes the index robust to a statutory change that happened in the middle of the corpus, a problem that does not arise in web search. §5.6 measures what it is worth: −0.1205 nDCG@10 when removed, ten times the next largest arm.

Explanation as a contract, not a feature. `ScoreExplanation` is part of the frozen interface in `nyaya/contracts.py`, so every result decomposes into per-zone contributions, top terms, proximity, authority and penalty. That is what let three people build against each other's components in parallel and debug ranking disagreements by reading a decomposition instead of guessing.

The ablation ladder is a flag, not six implementations. B0 → S1 → S2 → S2′ → S3 → S4 differ by one constant or one branch each, at the cost of carrying a baseline we will never ship — and the payoff is that every row of §5 is attributable to exactly one idea.

Vectorless RAG with a verifier, shipped extractive. The brief reuses the index we already measured rather than adding an embedding store, and wraps the composer so nothing can be asserted that the corpus does not contain. §5.5 gives the verifier's measured reach, including what it misses.

---

5 Evaluation

Everything is measured through one entry point, `search(query, k, mode)`, so the six systems are genuinely comparable.

Mode	What it is
B0	flat lnc.ltc tf-idf, plain tokenizer, no zones, no authority
S1	B0 + statute canonicalisation + legal stop list
S2	S1 + weighted zone scoring
S2′	S1 + BM25F instead of cosine
S3	S2′ + authority from plain (unweighted) PageRank
S4	S3 + treatment-aware g(d) + proximity — the full system

5.1 Judged queries

25 hand-written queries spanning constitutional, criminal, service and commercial matters, each with a stated information need; **5 are held out** for parameter tuning and the **20** reported here are the test split. Results from all six systems were pooled to depth 20, and **373** judgments were graded 0–3 by a law-reading member of the team who could not see which system had retrieved what. The qrels and the query set live in `eval/queries/`.

Two cutoffs, one metric, so the cutoff is in the name. Average precision is averaged over the whole ranking a harness submits, so its cutoff is that harness's run depth. This table retrieves 20 and reports **MAP@20**; the tuner (§2.10) and the ablation table (§5.6) score at k = 10 and report **MAP@10**. Both call the same `average_precision` in `nyaya.eval.metrics`, and AP@10 ≤ AP@20 by construction because recall is capped at ten results — so the two figures are one metric at two depths, not a contradiction.

Metric	B0	S1	S2	S2′	S3	S4
P@5	0.790	0.860	0.900	0.960	**0.970**	**0.970**
P@10	0.745	0.855	0.850	0.910	**0.920**	**0.920**
R@20	0.625	0.690	0.645	0.767	**0.774**	0.771
MAP@20	0.537	0.597	0.581	0.707	**0.723**	0.721
nDCG@10	0.610	0.642	0.671	0.798	**0.883**	0.879

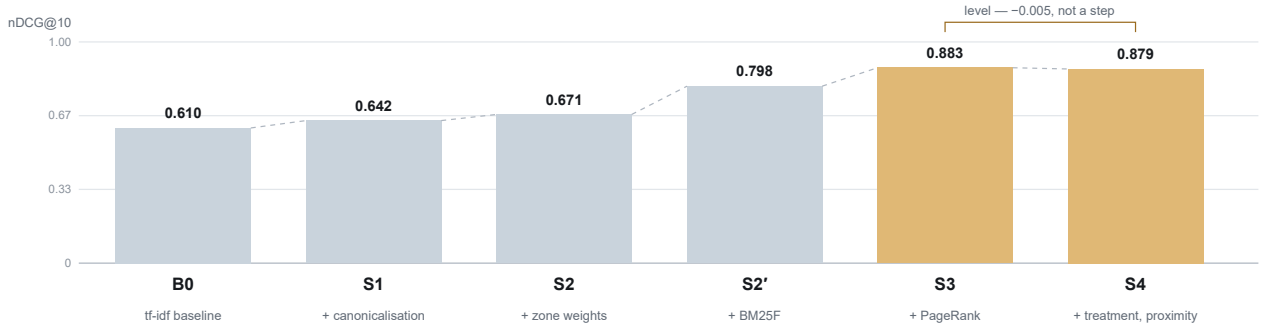


Figure 2 — the ablation ladder, nDCG@10 on the 20 judged test queries. Each step adds exactly one idea to the one before it. The two largest gains are BM25F (S2′, +0.127) and treatment-free authority (S3, +0.085). **S4 does not beat S3**: the gap is −0.005 and we mark it as level rather than drawing a step the data does not support — what S4 buys is not recommending overruled law, which no relevance-pooled metric can see. S1 → S2 raises nDCG@10 while *lowering* R@20 and MAP@20 (§5.1); that row is kept for the same reason.

We report the shape of the ladder, including the steps that do not move. BM25F is the largest single win (S2 → S2′, +0.127 nDCG@10, +0.126 MAP@20, from one change of scorer) and authority is the second (S2′ → S3, +0.085, on a metric that rewards putting the right judgment *first*). **Zone weighting alone goes backwards on recall**: S1 → S2 raises nDCG@10 but drops R@20 from 0.690 to 0.645 and MAP@20 from 0.597 to 0.581, and we keep the row rather than quietly dropping the system that caused it.

S4 does not beat S3 here and we do not claim it does. The gap is 0.005 nDCG@10 and 0.002 MAP@20 in S3's favour — noise at 20 queries. The honest reading is that what treatment-aware authority buys is *not recommending law that has been overruled*, and no precision- or recall-at-k measure over a relevance pool can see that, because a dead precedent is still topically relevant and will still be graded relevant. The value appears in §3.4, as 91 judgments demoted and badged, not in this table.

The end-to-end gap is significant, and we tested it rather than asserting it. S4 over B0 on nDCG@10 is +0.2686 (0.879 against 0.610): **17 queries improve, 0 tie, 3 get worse**, sign test p = **0.0026**, 10,000-iteration bootstrap 95% CI on the mean difference **[+0.1623, +0.3781]**, excluding zero. MAP@20 agrees (15/0/5, p = 0.0414) and P@10 too (10/8/2, p = 0.0386). At 20 queries a few points of mean difference would not be evidence on its own; this is what makes the headline gap a result rather than a hope, and why we decline to read the 0.005 gap.

5.2 Citation recovery — an automatic track with no human judge

Take an indexed judgment, hide every case it actually cites, use the first 20% of its paragraphs as the query, and ask how many of the hidden citations come back. **198** such queries, judgments from 2001 on with at least three indexed gold citations, seven systems including an authority-only control:

	B0	S1	S2	S2′	S3	S4	G0
R@10	0.1465	0.0603	0.0782	0.1571	**0.1760**	0.1734	0.0115
R@50	0.3003	0.1539	0.1559	0.3067	0.3392	**0.3404**	0.0352
MRR	0.2231	0.1152	0.1456	0.2872	**0.3084**	0.3069	0.0340

S3 and S4 are level here too, as on Track A, and for the same reason. Four things have to be said next to those numbers rather than in a footnote.

45.2% of the gold is unretrievable by construction. Of 28,028 resolved citations only 15,358 (54.8%) survived the ≤200-per-year sample, so recall here measures our ranking *and* our sampling together. It is a lower bound in a second way too: being cited is not the same as being the only relevant precedent, so a system returning an excellent *uncited* case is scored wrong for it.

The gold set is built from citations and authority ranks by citations, so S3 and S4 have a structural advantage. That objection is real, so we bounded it rather than argued with it. **G0** ranks by `g(d)` alone and never reads the query at all; whatever it scores is the floor the authority-bearing systems must be read against, and in every build so far it has been a small fraction of S3. Authority alone is close to useless on this track — it is the text scorer doing the work and authority ordering what the text scorer found.

The query must not leak the answer. 5,613 citations, 3,482 case names and 3,565 cue sentences are masked from every query, and the masking is audited for *distinctive* leakage (df ≤ 100) rather than for any shared word: the audit finds **0.00%, 0 of 1,222 gold pairs**. The 6.4% of pairs sharing an ordinary word like "income" are not the answer leaking and cannot be stripped without destroying the query.

S1 and S2 fall below the baseline here and we leave that visible — stop-listing plus zone weighting costs recall on long, paragraph-shaped queries, and BM25F at S2′ recovers it and more. Finally, **wiring the champion lists into live search cost this track nothing**: across all six systems not one query lost R@10 or R@50. A speed optimisation not paid for in recall is rare enough to report.

5.3 Operator correctness

94 checks — 10 Boolean, 14 phrase, 50 `/n` across $k \in \{1, 3, 5, 10, 25\}$, 10 `/s` and 10 `/p` — run over 1,200 judgments sampled every 11th document so the span is 1950–2025. Each is checked two ways.

Against an independent naive implementation: 94 of 94, 100.00%. The token stream is rebuilt from raw text and scanned with nested loops — no postings, no skip pointers, no numpy merge, no planner — and the two result *sets* must be identical, not similar. Every family passes: Boolean 10/10, phrase 14/14, each `/n` width 10/10, `/s` 10/10, `/p` 10/10. Pass-or-fail, and the one number in the project allowed to be boring.

Against a raw-text regex: 2,740 of 2,888 document decisions agree, 94.88%, and all 148 differences are classified with **zero unexplained**: 135 stemming conflation, where the index matches `convicted` for `convict` because Porter says so (including honest over-conflation — `negligence → neglig` also matches `negligible`); 9 the regex's own over-match, where a crude `bail\w{0,3}` catches *bailiff*; 3 token-count differences over numerals; 1 adjacency across OCR junk. We give both figures because the per-*check* regex number (27 of 44) overstates the disagreement badly — a check differing on one document of 1,092 counts the same as one differing on all of them.

5.4 Efficiency

We report the algorithmic reduction, not an end-to-end speed-up, and the reason matters. Three runs of *identical code* gave monotonically falling wall clock as the operating system's page cache warmed — B0 went 6.18 → 5.22 → 4.00 ms — while the algorithmic columns were identical every time. Any latency ratio we quoted would be measuring the page cache rather than the champion lists, so we do not quote one. What the optimisation does is exact and reproducible:

Quantity, 20 queries, k = 10	Exhaustive	Tier 1	Reduction
candidates, p50	12,548	1,059	11.8× fewer
documents scored, p50	8,032	1,030	7.8× fewer
documents scored, whole set	159,328	17,718	**8.99× fewer**

Latency is reported as a range rather than a ratio: **4–7 ms warm p50 across all six systems**, with a cold start of 1.2–1.5 s to memory-map the stores and about 1 s for the first query. Two exact structures sit underneath: df-ordered Boolean planning, and a heap for the top k instead of a full sort. Both are lossless. The first is measured in §2.6 — over eight AND chains **145,921 → 69,527** postings entries compared, **2.10× less work for identical answers**. **Index elimination** removes 16.5% of the documents scored (132,206 → 110,396) for top-10 identical on 16 of 20 queries and a mean overlap of 0.985.

The two-tier index is the only non-safe optimisation in the system, so its fidelity is printed beside its gain. On the shipped path, S4, 20 queries:

	value
documents scored	159,328 → 17,718 (8.99× fewer)
top-10 identical	**18 / 20**, mean overlap 0.990
rank 1 unchanged	**20 / 20**
fell through to tier 2	0 / 20
worst case	**9 / 10**, twice in twenty (*unlawful activities prevention*)
cost on the judged set	**zero** — nDCG@10, MAP@20 and P@10 identical to four decimals with tiers on and off, in all three authority modes

Our offline harness reports the same optimisation as 12/20 identical with a 3/10 worst case. The difference is a real path difference, not noise: the harness builds candidates from `scoring_terms`, so *appellate jurisdiction of the supreme court* collapses to the single term `suprem` and tier 1 is that one term's 400 champions, whereas `search()` builds from `free_terms` and tier 1 is a union of six. We report the shipped numbers *and* name the harness arm rather than quietly keeping the better of the two. Across three runs the tier wall-clock gain ranged **3.2×–4.2×**, which is why we lead with the document count.

5.5 Precedent Brief verification

The Precedent Brief ships extractive, and the output says so on every brief. No LLM API key was available in this environment — `ANTHROPIC_API_KEY` and `OPENAI_API_KEY` are both unset and the `anthropic` SDK is not installed — so the abstractive composer, although written and prompt-complete, **has never been executed**. We would rather ship a working extractive feature with an honest boundary than claim an LLM result we cannot reproduce. Across six real briefs: 6 sentences drawn from 8 paragraphs across 4–5 judgments, every sentence carrying a paragraph id, 0 flagged, 200–1,000 ms.

The verifier is measured against deliberate fabrications, not asserted. 36 real extractive sentences score a tf-idf cosine of 0.9924–1.0000 against the paragraph they cite; 18 fabrications written to defeat it score 0.0866–0.8451. At the shipped threshold of 0.20 it catches **4 of the 18, with 0 false alarms on the 36 real sentences** — and we state that as a limitation rather than a feature. It reliably catches the *mechanical* failures (no citation, or a citation outside the evidence pack) but not a confident on-topic falsehood: *"The right to privacy was expressly held to be subordinate to the right to property"* scores 0.8451 and passes. **A bag-of-words cosine is a topicality check, not an entailment check.** We did not raise the threshold to 0.90 to flatter the number, because that would reject a genuine paraphrase the moment the abstractive composer is enabled; a test pins the limitation, and says in its docstring that if it ever fails the verifier improved and these numbers need replacing.

5.6 Ablations — what the domain engineering is actually worth

One flag off at a time, 20 test queries, mode S4. This is the clearest evidence in the project that the *vertical* work, rather than the generic IR machinery, is what earns the result.

Arm	What is off	nDCG@10	Δ	MAP@10	P@10	Vocabulary
full	— the shipped analyser	**0.8786**	—	0.5473	0.933	544,787
nostem	Porter stemming, both sides	0.8669	−0.0117	0.5137	0.873	618,852
nocanon	statute canonicalisation	**0.7582**	**−0.1205**	0.4631	0.783	539,096
noinherit	act inheritance, body zone	0.8670	−0.0116	0.5473	0.933	544,843

Statute canonicalisation is worth ten times what stemming is worth, and the loss lands exactly where the mechanism is supposed to be working — on the statute-anchored queries. q23 collapses from 0.7761 to **0.0000**, q14 from 0.9545 to 0.2440, q15 from 0.8128 to 0.4053. Without `ipc§302` as a single token those queries have nothing to match on, which is the vocabulary-mismatch problem of §1 reappearing as a measurement. Stemming, by contrast, buys +0.0117 nDCG@10 and a **14.5% smaller vocabulary** — useful, and an order of magnitude less important.

Two honesty notes on this table. `noinherit` is a **partial** ablation: it is query-side only, because the statutes zone is synthesised from tokens computed at ingest time, so a complete arm needs a re-ingest — the code says so where it is implemented, and §6 describes the silent no-op we had to fix before this row meant anything at all. And the vocabulary column should not be read below about 100 types: the build is not bit-deterministic, so `noinherit`'s 56-type gap from `full` is noise rather than signal.

---

6 Limitations and next steps

The sample caps recall, and we do not hide it. We index 13,878 of 37,210 judgments. A precedent that was not sampled cannot be retrieved, which puts a hard ceiling on §5.2 and depresses recall on any query whose best answer happens to fall outside the draw. The citation *graph* covers all 37,210 de-duplicated nodes, so authority is computed over the full universe even where we lack the text — but that also means some high-authority nodes are unsearchable.

The treatment classifier cannot read a denial. It matches surface cues, so "We are **unable to hold** that the decision in *Rajesh v. State* **stands overruled**" contains the cue for an overruling inside an explicit refusal to overrule, and is mis-labelled. A case that was upheld then loses three quarters of its `g(d)`. On our hand-labelled sample of **40** edges the classifier is right **92.5%** of the time (37 of 40), and **all three** of its errors are an edge the citing court merely *mentioned* being read as something stronger on a nearby cue. Per class it is perfect on *distinguished* and on *mentioned* precision; *overruled* precision is 80% and *mentioned* recall is 77%, so everything it loses, it loses **upward** — into a label that demotes. That asymmetry is the worst possible direction for this error and it is why the next item on the roadmap is negation scope. The fix is a negation and modality *scope* around the cue, not a longer phrase list; until it lands, the flag is visible in the interface together with the sentence that caused it, so a user can overrule the system.

Bench size is sometimes a fallback rather than a reading. Re-parsing the CORAM line moved the recorded bench on 488 documents: 258 up, but **230 down, 184 of them to 1**, where the re-parse refused to validate a bench line it could no longer read and the system fell back to the metadata's single authoring judge. `G_BENCH_WEIGHT` then reads those 184 as single-judge benches and weights them accordingly. We prefer "no claim plus a declared fallback" to "a confident wrong claim", but it is a real hole in a parametric field that `bench:>=5` filters on.

OCR puts a visible floor under the bench parser. *Maneka Gandhi* (1978) is a seven-judge bench and our index records it as **1**, because its bench line OCRs as `M. H. BEG, c. J.` with the other six judges on lines the parser never reaches. It is cosmetic for ranking — one document's bench term — but it is visible in the interface, and it is the honest face of a corpus whose older half is scanned paper.

Derived stores can silently go stale, and ours did. Two fixes were correct in the code and had still not reached the ranking, because a store downstream of them was older than the corpus it was derived from: Puttaswamy served `bench 1` from an S2 built before the bench reparse, and *Maneka Gandhi* was still flagged `overruled` from an authority store built twenty-five minutes before the re-classification pass that fixed it. Nothing was wrong with either algorithm; the pipeline simply has five stores and no staleness check between them. Of every limitation in this section it is the most ordinary and, in a multi-stage index, the most likely to recur — so the roadmap carries a build-stamp check and we say plainly that we were caught by it.

Bench size is sometimes a fallback rather than a reading, and it is not cheap. Re-parsing the CORAM line moved 488 documents: 258 up (163 of them from 1, so the re-parse is net-correcting) but **230 down, 184 to 1**, where it refused to validate a bench line it could not read and fell back to the metadata's single authoring judge. 999 of 13,942 documents (7.2%) record bench 1, roughly a quarter of them parse failures. The cost is quantifiable, which is what makes this a limitation rather than an untidiness: `bench_norm = min(bench,13)/13` with `G_BENCH_WEIGHT = 0.25` means each missing seat costs **0.0192 of g(d)**, so a five-judge bench read as 1 understates `g` by **0.077** and a thirteen-judge bench by **0.231** — against an *entire* overruled penalty of 0.289 on Pune Municipal Corporation. One misparsed Constitution Bench moves authority by the same order as being overruled. *Maneka Gandhi* (1978) is the visible case: seven judges recorded as **1**, because its bench line OCRs as `M. H. BEG, c. J.` and the parser never reaches the other six.

Derived stores can go stale silently, and ours did, four times. Two fixes were correct in code and had still not reached the ranking because a downstream store was older than the corpus: Puttaswamy served `bench 1` from a parametric index built before the bench reparse, and *Maneka Gandhi* was flagged `overruled` from an authority store built twenty-five minutes before the re-classification that fixed it. A third was a stale *assertion* rather than stale data — `runs/efficiency.json` still said "S5 is NOT yet wired into `search()`" after that became false, and it would have entered this report as a falsehood. The sharpest was an ablation arm that *silently did nothing*: the act-inheritance arm reproduced the shipped numbers to five decimals, which reads as "inheritance does not matter" and meant the flag was never off. Two independent causes — a `flag: bool = INHERIT_ACT_DEFAULT` default argument is bound at **import**, so reassigning the module constant changed nothing; and the builder fans out over `ProcessPoolExecutor`, which on Windows spawns by re-importing, so a parent-process flip never reaches a worker. Fixed with call-time resolution and an environment variable, and pinned by a test that runs a **real child process**, because no in-process test can see that class of bug. The arm is still a *partial* ablation and says so in the code.

All four share one lesson, and it is the one we would most want a reader to take from this project: **in a multi-stage index pipeline, a correct fix that never reaches the derived stores is indistinguishable from no fix at all.** No unit test would have caught any of them, because each was a *build* failure wearing the costume of a *result*.

The index build is not bit-deterministic — two builds from identical inputs gave vocabularies differing by 36 and 56 types, so §5.6's vocabulary column should not be read below about 100.

Name-based citation resolution is the weakest link in the graph. Neutral and SCR citations resolve cleanly; a Jaccard threshold over party names does not, and Indian case names repeat heavily (*State of Maharashtra v.* …). Every edge records `resolved_by`, so the damage is measurable — but it is real.

Extraction loss. Older judgments are scans, and pages under 200 words are treated as images and dropped, biasing the indexed corpus towards the typeset era within each year's quota.

One judge, one assessor pool. The relevance judgments come from one person on the team: blind to system identity, but not independent, and 20 queries is a small test set. We answered that with a sign test and a bootstrap rather than a disclaimer — the B0→S4 gap survives both and we read it as real; the 0.005 between S3 and S4 does not, and we do not.

The Precedent Brief's verifier is a topicality check, not an entailment check (§5.5), and the abstractive composer has never been run, because no model key was available here.

No High Courts, no statutes as documents, though precedent in practice runs through both. **And one judge, one assessor pool** — see above.

Roadmap

Horizon	Work	Why it is next
Next	Negation and modality scope around treatment cues — a dependency window, so "unable to hold" cancels the match	The largest source of wrong labels, and a wrong label demotes good law
Next	A build-stamp check between the five derived stores	§6: all four of our worst bugs were a store older than its input
Then	Supervised treatment classifier over the hand-labelled set, keeping the cues as features	Turns our test set into training data, with a fresh held-out set
Then	Lift the per-year cap, first for the years the query set touches	Directly raises the §5.2 ceiling, which is 45.2% unretrievable today
After	High Court judgments, bare statutes, and a *cited-by* browse over the treatment graph	The graph is already the right shape; this makes it a tool, not a demo
After	Learned ranking over the existing features, once the judged set can support it	α and β are hand-set; §2.10 shows 20 queries cannot yet fit them

7 Work division

Interfaces were frozen at kick-off in `nyaya/contracts.py` with the constants in `nyaya/config.py`, and the three of us built against them in parallel. Ownership is file-level: nobody edits a module they do not own.

Rakshit (M1) — download, PDF extraction, zoning, paragraph segmentation and the extract log; the SPIMI build, zoned positional postings, skip pointers and the memory-mapped S3 store; proximity operators `/n` `/s` `/p` and phrase matching; the proximity contribution and snippets; the cue-phrase treatment classifier; the evaluation harness and its metrics (P@k, R@k, MAP@k, nDCG@k) and the relevance judging; video production.

Suvrat (M2) — the metadata join, universe and manifest, and the stratified sample; the parametric index (S2) and the champion lists and tiers (S5); the df-ordered Boolean planner, index elimination and filter folding; citation extraction and resolution, the treatment-weighted PageRank, `g(d)`, the net score and the heap; the citation-recovery and efficiency tracks; report assembly.

Sumedha (M3) — the paragraph store (S1) and collection statistics; the query language, parser, normaliser and root expansion; lnc.ltc cosine, weighted zone scoring, BM25F and "why this result"; the statute canonicaliser, the IPC↔BNS crosswalk, and the Precedent Brief with its verifier; query-set curation and the operator correctness tests; the CLI, the Streamlit page, the diagrams and script.

Shared and append-only: `config.py`, `contracts.py`, `utils.py`, `README.md`. Every component has a readable test in `tests/` and can print an intermediate output on demand — postings, plan, weights, scores — which is both a debugging contract and what the demo video shows, end to end, as live terminal output.

AI-use declaration

This project was built with AI coding agents. The assignment permits that on condition it is declared, so it is declared here specifically.

What the AI did. Most of the implementation code in `nyaya/`, the tests in `tests/`, the Remotion video project in `video/` and the first draft of this report were written by Claude Code (Anthropic) agents under our direction, in parallel, each confined to the files its owner is responsible for.

What we did. We chose the problem and the track. We designed the architecture: the six zones and their weights, the mode ladder B0→S4, the treatment-weighted authority model and the decision to separate edge weight from status penalty, the net-score formula, and the decision to build vectorless RAG with a verifier rather than add an embedding store. We wrote and froze the interface contracts in `nyaya/contracts.py` and the constants in `nyaya/config.py` before any implementation began, and the agents were required to build against them. We set the corpus policy — the universe/indexed-subset split, the ≤200-per-year stratified sample, the seed — and accepted its cost to recall. We designed the evaluation: which systems to compare, which metrics, pooled judging, the two automatic tracks, and the rule that an ablation which does not help gets reported anyway. We wrote the relevance judgments ourselves; no model graded its own output. We reviewed the generated code, ran it on the real corpus rather than on stubs, and are answerable for every claim in this report.

What the AI did not do. It did not choose the architecture, the corpus policy or the evaluation design, and it did not produce the relevance judgments. It did not invent any number in §5: every figure there is the output of a command named in Appendix A, run on the real corpus. Where a measurement was unavailable the report carried a visible placeholder until the command had been run — none remain.

Tooling. Claude Code (Anthropic) for implementation, tests, the video project and report drafting. Standard libraries only for the IR core: NumPy for the postings arrays and the power iteration, pandas and PyArrow for the metadata, a PDF text extractor for ingest. No search engine, no graph library, no embedding model, no ranking library — the index, the merges, the scorers and the PageRank are ours.

References

1. Manning, Raghavan & Schütze, *Introduction to Information Retrieval*, CUP 2008 — SPIMI, skip pointers, lnc.ltc, zone scoring, champion lists, tiered indexes, index elimination.
2. Robertson & Zaragoza, *The Probabilistic Relevance Framework: BM25 and Beyond*, Foundations and Trends in IR 3(4), 2009 — BM25 and the BM25F field-combination rule.
3. Page, Brin, Motwani & Winograd, *The PageRank Citation Ranking: Bringing Order to the Web*, Stanford InfoLab, 1999.
4. Järvelin & Kekäläinen, *Cumulated Gain-based Evaluation of IR Techniques*, ACM TOIS 20(4), 2002 — nDCG.
5. Porter, *An Algorithm for Suffix Stripping*, Program 14(3), 1980.
6. Indian Supreme Court Judgments, AWS Open Data Registry, CC-BY-4.0.
7. The Bharatiya Nyaya Sanhita, 2023 (in force 1 July 2024) — the IPC↔BNS section mapping.

Appendix A — reproducing the results

Every number in this report comes from one of these, and each writes its result into `runs/`. Set `PYTHONIOENCODING=utf-8` first, or the statute tokens print as boxes.

```
# the build, in order (see README.md for timings)
python -m nyaya.metadata all                          # universe + manifest
python -m nyaya.ingest download                       # 7.7 GB of judgment PDFs
python -m nyaya.ingest build                           # extract + zone -> judgments.jsonl
python -m nyaya.index.positional build --both          # S3 and B0's index
python -m nyaya.index.paragraphs --build               # S1
python -m nyaya.index.parametric build                 # S2
python -m nyaya.authority.citations build              # S4 edges
python -m nyaya.authority.treatment label              # label those edges
python -m nyaya.authority.pagerank build --weighting treatment
python -m nyaya.index.tiers build                      # S5
python -m pytest tests -q                             # all component tests
python -m nyaya.cli --smoke                           # end-to-end smoke test

python -m nyaya.index.stats --no-growth               # §1, §2.3  corpus_stats.json
python -m nyaya.authority.citations stats             # §3.2    citations_stats.json
python -m nyaya.cli --treatment-audit --only-errors   # §3.3, §6
python -m nyaya.authority.pagerank compare            # §3.4    the g(d) that moved
python -m nyaya.rank.tune --report-test               # §2.10   tuning.json
python -m nyaya.eval.run                              # §5.1    trackA.json
python -m nyaya.eval.citation_recovery run --n 198    # §5.2    citation_recovery.json
python -m nyaya.eval.citation_recovery report
python -m nyaya.eval.operators --brute-force          # §5.3    operators.json
python -m nyaya.eval.efficiency run                   # §5.4    efficiency.json
python -m nyaya.eval.efficiency report
python -m nyaya.cli --brief "<query>"                 # §5.5
python -m nyaya.rank.ablate --run                     # §5.6    ablations.json
python -m nyaya.query.planner --compare-suite         # §2.6    df-ordering, 8 chains
```

`--n 198` on the Track B line is not decoration: `run` reuses `runs/citation_recovery_queries.json` when it holds at least *n* queries and re-samples otherwise, and the figures in §5.2 are the first 198 of that saved set. Ask for 200 and you get a slightly different sample and R@10 within 0.002 of the table — correct, but not the same run.

`config.SAMPLE_SEED = 358` fixes the corpus draw, so a rebuild reproduces the same indexed subset — though not byte for byte: see the determinism note in §6. The two `report` subcommands re-print a finished run without recomputing it, which is what the demo video records; if the derived stores have been rebuilt since, run the matching `run` subcommand first or the report will print stale numbers.

Appendix B — query language reference

Form	Meaning
`murder bail`	free text; fast path through tier 1
`"common intention"`	exact phrase
`privacy /5 dignity`	within five words
`privacy /s dignity`	same sentence
`privacy /p dignity`	same paragraph
`bail!`	root expansion by dictionary prefix
`title:(privacy)`	restrict to one zone
`year:>=2000`, `bench:>=5`	parametric filters
`judge:chandrachud`, `act:IPC`, `type:criminal`	parametric filters
`X AND Y`, `X OR Y`, `X NOT Y`	Boolean; `NOT` is a set difference, never a bare negation